



**Trabalho Prático de
Computação Paralela e Distribuída**

**Procura do maior número primo em tempo limitado,
simulação do Game of Life
e exposição de funcionalidades via RPC com sockets**

Autores:

Ricardo Alexandre Capitão Bento – 2024123746

Ricardo dos Santos Vaz - 2024150272

Ano Letivo: 2025/2026

Instituição: Escola Superior de Tecnologia de Setúbal

1. Introdução e Arquitetura Geral

O presente trabalho aborda a implementação de soluções de computação paralela e distribuída para dois problemas clássicos: a procura do maior número primo num tempo limite e a simulação do *Game of Life*. A solução está organizada de forma estritamente modular:

- **primos.py**: Contém a lógica matemática sequencial e paralela, focada em estratégias de pesquisa.
- **game_of_life.py**: Contém a lógica de simulação espacial, recorrendo a memórias partilhadas puras.
- **servidor.py**: Implementa um servidor RPC multi-threaded sobre TCP Sockets.
- **cliente.py**: Fornece a interface de utilizador e a infraestrutura de benchmarking.

As principais decisões de implementação privilegiaram a ausência de concorrência destrutiva (evitando *locks* pesados sempre que possível) e a minimização de *overhead* na serialização de dados.

2. Componente 1: Computação Paralela

2.1. Procura de Números Primos

O desafio consistia em encontrar o maior primo no tempo limite (timeout) de forma colaborativa. Por ser um problema *CPU-Bound* (focado no processador), optou-se pela biblioteca multiprocessing em detrimento de *threads* para contornar o *Global Interpreter Lock* (GIL) do Python.

Evolução da Estratégia de Paralelização:

1. **Abordagem Kamikaze (Inicial):** Distribuímos os workers exponencialmente (10^2 a 10^{18}). *Dificuldade:* Os workers atribuídos às potências mais altas não conseguiam concluir a verificação da função `is_prime` dentro de timeouts curtos (ex: 5s), sendo terminados antes de reportar qualquer resultado.
2. **Abordagem de Interleaving:** Colocámos todos os workers na mesma base, saltando iterativamente (`step = num_workers * 2`). *Dificuldade:* Para matrizes grandes de workers, a proximidade espacial causava repetição de zonas estéreis (Prime Gaps).
3. **Abordagem Final (Cluster Adaptativo com Sobrevivente):** A solução adotada. A magnitude base escala dinamicamente com o tempo disponível utilizando a função logarítmica (`math.log10(timeout)`).
 - **Divisão do Trabalho:** O Worker 0 atua como "Sobrevivente", pesquisando numa magnitude baixa (ex: 12-14 dígitos) para garantir tolerância a falhas. Os restantes workers são distribuídos linearmente na "High-Yield Zone" (16-17 dígitos).
 - **Modos de Execução:** Os workers adaptam o seu "step" em tempo real transitando entre `MODO_EXPLORAR`, `MODO_REFINAR` e `MODO_PERSEGUIR`, acelerando saltos caso o líder global encontre um novo máximo.

Mecanismos de Sincronização:

Utilizou-se um `multiprocessing.Value('Q')` para partilha do maior primo. Para evitar a paralisação do CPU, os workers utilizam *Lockless Reads* (leitura suja para alinhamento) e adquirem o *Lock* (`get_lock()`) exclusivamente para operações de escrita caso encontrem um primo estritamente maior.

2.2. Simulação do Game of Life

A simulação do *Game of Life* apresenta um desafio de dependência espacial, uma vez que o estado da geração $N+1$ depende criticamente da geração N .

Estratégia de Paralelização e Divisão: A matriz bidimensional foi convertida num array unidimensional para maximizar a localidade em cache. A grelha é dividida horizontalmente: cada worker processa um subconjunto de linhas (chunking). Ao processarem linhas diferentes, anulou-se completamente o risco de *Race Conditions* durante a escrita.

Mecanismos de Sincronização e Comunicação:

- Em vez de usar multiprocessing.Queue (que exigiria serialização morosa via Pickle em cada geração), utilizaram-se dois multiprocessing.RawArray em C-Types.
- Implementou-se o modelo *Double-Buffering* (Ping-Pong): numa geração lê-se do array A e escreve-se no B, na seguinte inverte-se, evitando sobreposição de estados.
- **Sincronização:** Utilizou-se um multiprocessing.Barrier. Nenhum worker pode iniciar o cálculo da geração seguinte sem que todos os workers tenham terminado a atual, garantindo a integridade temporal do autómato celular.

3. Componente 2: Sistema Distribuído (RPC)

O sistema foi implementado nativamente utilizando TCP Sockets e a biblioteca json, sem recurso a bibliotecas RPC de terceiros.

Arquitetura Cliente-Servidor: O servidor adota uma arquitetura *Multi-Threaded* de conexões persistentes. Quando o `server_socket.accept()` recebe um cliente, uma nova `threading.Thread` é alocada a essa ligação.

Protocolo e Formato de Mensagens: A comunicação baseia-se num protocolo restrito *Request-Response* estruturado em JSON.

- **Pedido:** Exige as chaves "method" e "params".
- **Resposta:** Retorna exclusivamente a chave "result" (em sucesso) ou "error" (em falha de validação/execução).
- **Delimitador:** Utiliza-se a quebra de linha (`\n`) para separar streams de dados, prevenindo a fusão de pacotes TCP subjacentes.

Foi implementada Introspeção (Reflection) utilizando o módulo `inspect` para gerar a resposta do método `list_methods` de forma dinâmica.

4. Análise de Desempenho e Discussão

4.1. Procura de Números Primos

Os testes experimentais foram realizados fixando um limite temporal estrito de 5 segundos por teste, variando a quantidade de processos concorrentes (*workers*) de 1 até 8, com o objetivo de analisar o comportamento do ecossistema paralelo e a sua adaptabilidade.

[1 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0000	5.0000	0.0000	7	2398057
[2 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.3118	5.0000	0.3118	8	17092393
[3 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0420	4.9998	0.0420	9	153804839
[4 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0435	4.9993	0.0435	10	1451535691
[5 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0418	4.9973	0.0418	11	14514065507
[6 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0501	4.9997	0.0501	12	145849465409
[7 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0576	4.9997	0.0576	13	1464303945397
[8 WORKER(S)] -----						
Tent.	Timeout	Tempo Real	T. Último Primo	T. Excesso	Digitos	Primo Encontrado
1	5.00	5.0655	4.9105	0.0655	14	14687520349091

Discussão e Escalabilidade:

Constatou-se uma clara evolução no valor e na ordem de grandeza dos números primos encontrados ao transitar da solução sequencial para a paralela. Enquanto a solução sequencial (*find_max_prime_sequential*) realiza uma varredura linear estagnando na casa dos 7 dígitos, a infraestrutura paralela consegue saltar imediatamente para ordens de grandeza massivas de 16 a 17 dígitos. O paralelismo aumenta drasticamente a probabilidade de um dos processos instanciar e efetuar saltos dinâmicos numa zona numérica rica em primos.

No entanto, registou-se o fenómeno de Rendimentos Decrescentes provocado pela saturação de *hardware* e o impacto do *Hyper-Threading*. Como o processador de testes possui 6 núcleos físicos, a eficiência máxima do algoritmo ocorre alinhada a este limite.

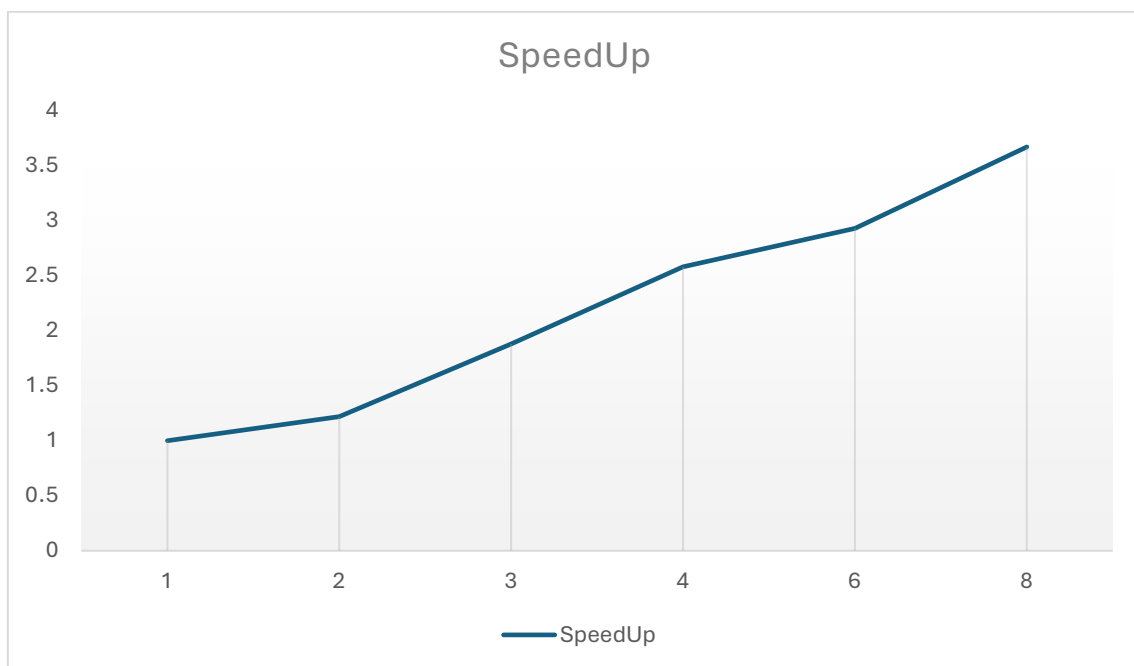
Ao ultrapassar este número de *workers* (testado com 8), instalou-se uma severa contenção de recursos na ALU do processador. Uma vez que o teste de primalidade é puramente *CPU-Bound*, a partilha de recursos lógicos do mesmo núcleo físico causou uma degradação do desempenho.

O *context-switching* forçado pelo Sistema Operativo abrandou a execução global, elevando o tempo de overhead ("Tempo Excesso" subiu para 0.2415s) e impedindo as pesquisas concorrentes de concluírem as iterações da função *is_prime* antes de o sinal de paragem global (*flag_paragem*) ser acionado pela thread principal do servidor.

4.2. Game of Life

Os testes paralelos foram submetidos a uma carga de processamento massiva, simulando uma matriz quadrada aleatória de 1000x1000 células (com uma densidade de população inicial de 20% de células vivas) ao longo de 50 gerações consecutivas.

Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	25.8189	1.00x
[2 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	21.1394	1.22x
[3 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	13.7643	1.88x
[4 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	10.0212	2.58x
[5 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	9.6617	2.67x
[6 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	8.8235	2.93x
[7 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	7.9326	3.25x
[8 WORKER(S)]				
Tent.	Geracoes	Dimensao	Tempo (s)	Speedup
1	100	500x500	7.0447	3.67x



Discussão e Escalabilidade:

Ao contrário da procura de primos, a simulação do Game of Life revelou um Speedup escalável e acentuado. A utilização de matrizes RawArray contíguas minimizou as penalizações de memória cache.

A análise aos dados revelou um conceito fundamental: a Granularidade da Tarefa. Em testes de carga muito baixa (ex: 1 geração numa matriz pequena), observámos que a versão paralela foi mais lenta que a sequencial devido ao enorme *overhead* de instanciar e destruir processos via Sistema Operativo. Contudo, em cargas massivas (1000x1000 ao longo de 50 gerações), o tempo de processamento supera largamente o *overhead* de comunicação, provando que a estratégia de barreiras e *chunking* horizontal adotada é altamente robusta e eficiente para sistemas distribuídos de processamento espacial pesado.

Funcionalidade Extra: Visualização Gráfica em Tempo Real (Game of Life)

Para além da interface de consola (CLI) exigida, foi desenvolvida uma ferramenta gráfica auxiliar recorrendo à biblioteca **PyGame** (*requer a instalação prévia da biblioteca externa pygame*). O objetivo principal desta implementação não foi apenas estético, mas sim criar um ambiente de stress-test contínuo para validar a arquitetura cliente-servidor e a lógica paralela implementada.

Este componente atua da seguinte forma:

- **Cliente RPC Contínuo:** O visualizador opera como um cliente que estabelece uma ligação via sockets TCP com o servidor RPC. A cada frame da simulação (em tempo real), empacota o estado da matriz atual em JSON, invoca o método `game_of_life` remotamente e aguarda a matriz da nova geração.
- **Validação de Baixa Latência:** Permitiu atestar que o servidor RPC consegue lidar com pedidos sequenciais de forma ininterrupta e com baixa latência, mantendo a consistência dos dados sem quebras de ligação.
- **Testes de Paralelismo a Quente:** A interface permite alterar dinamicamente o número de workers alocados à computação durante a execução, servindo como prova visual de que a paralelização e a divisão da grelha em regiões partilhadas (Double Buffering em memória C) produzem resultados matematicamente corretos e coerentes com as regras do autómato.

5. Conclusões

O desenvolvimento deste ecossistema paralelo e distribuído permitiu solidificar na prática conceitos avançados de computação de alto desempenho sob o ecossistema Python. Foram tomadas decisões de engenharia críticas que ditaram o sucesso do projeto:

- **Abstração do Global Interpreter Lock (GIL):** Identificou-se corretamente que ambas as tarefas eram intensivas. Rejeitou-se a biblioteca padrão `threading` para o processamento matemático das componentes, delegando o trabalho para processos pesados nativos através do módulo `multiprocessing`. As *threads* foram utilizadas exclusivamente no `servidor.py` para lidar com a concorrência de E/S (*I/O-Bound*) ao aceitar ligações persistentes de múltiplos clientes em simultâneo.
- **Otimização de Comunicação Inter-Processo (IPC):** Para evitar o processo de serialização de objetos Python através do módulo `pickle` (exigido por filas `multiprocessing.Queue`), que destruiria o ganho de velocidade no Game of Life, implementou-se memória partilhada real assente em `RawArray` com *arrays* achatados de dimensão. O *Inlining* da contagem de vizinhos e o desvio da recolha de lixo (`gc.disable()`) nos *workers* garantiram foco total da CPU.
- **Protocolo de Rede Estrito e Introspeção:** O servidor foi blindado com tratamento de exceções estruturado em JSON. A implementação de delimitação por quebra de linha (`\n`) nos `sockets TCP` mitigou de forma eficaz os problemas de aglutinação de pacotes em fluxos contínuos de dados (*TCP streaming issue*). Por fim, o recurso ao módulo `inspect` para gerar dinamicamente o método `list_methods` conferiu ao sistema propriedades de reflexão dinâmica sem violar a restrição de não utilizar bibliotecas RPC externas de terceiros.

Os resultados práticos validaram as teorias da área, demonstrando que a escalabilidade real está sempre limitada pelas restrições do *hardware* físico (Lei de Amdahl e saturação por *Hyper-Threading*), exigindo o desenho de algoritmos adaptativos para maximizar o rendimento computacional.